Technical Design & Implementation Document: Voice AI Demographic Inference Service

1. Executive Summary & Problem Statement

Dialflo builds voice AI agents that handle inbound and outbound calls for logistics companies, coordinating deliveries, resolving exceptions, and communicating with drivers, dispatchers, and customers. To personalize these interactions seamlessly, our agents need to quickly infer caller attributes (specifically **gender** and **age bracket**) directly from streaming or chunked audio **without relying on any prior CRM data or historical contact history**.

Your task is to design, implement, and containerize a high-performance backend service that accepts audio streams, analyzes audio quality, performs machine learning inference, and returns a structured response matching a strict API contract under strict real-time constraints.

2. Core Constraints & Technical Requirements

- **Strict Latency:** Target end-to-end inference under **500ms** on a 5-second audio chunk.
- **Logistics Context & Noise Resilience:** Calls frequently contain heavy background noise (truck engines, warehouse machinery, road noise). The system must evaluate signal quality and degrade gracefully by flagging audio quality rather than failing silently.
- **Zero-Disk Privacy Compliance:** Caller audio is classified as Personally Identifiable Information (PII). **No audio can be saved to disk**, even temporarily (e.g., no writing to /tmp). All processing must occur strictly in-memory using byte buffers (io.BytesIO).
- **Portability:** The application must run via docker compose up with no external dependencies other than publicly available pre-trained model weights.

3. Foundational Knowledge & Concepts

To implement this service successfully, we established understanding across several fundamental domains:

- **Digital Audio Fundamentals:** Sound waves are digitized via **Sampling** (taking discrete snapshots of amplitude per second, standardizing telephony audio at 8 kHz or 16 kHz to match ML model expectations) and **Quantization / Bit Depth** (mapping voltage values to precise numerical scales).
- **PCM vs. Compressed Codecs:** MP3/WAV/OGG stream wrappers must be unpacked into uncompressed **Pulse Code Modulation (PCM)** arrays (represented in Python as 1D arrays of 32-bit floating-point numbers between -1.0 and +1.0).
- **Asynchronous Concurrency:** Audio decoding and ML matrix multiplication are heavy CPU-bound operations. Using FastAPI's async event loop requires careful thread offloading (e.g., asyncio.to_thread) to prevent blocking the single-threaded event loop during concurrent calls.

4. System Architecture & Modular Pipeline

To ensure a maintainable, high-performance architecture, the problem was decoupled into **5 isolated modules**, mirroring decentralized streaming pipelines (such as ROS 2 nodes):

HTTP/WS Request —► [1. In-Memory Ingestion] —► [2. Quality Gatekeeper] —► [3. ML Inference Engine] —► [4. Post-Processor] —► [5. FastAPI Router]

1. **Module 1: Audio Ingestion & Decoding** (app/services/audio_decoder.py)
    - *Input:* Raw binary network bytes (bytes via multipart/form-data or streams).
    - *Processing:* Wraps bytes in an in-memory io.BytesIO buffer, decodes compressed formats, downmixes stereo to mono, and resamples to **16,000 Hz**.
    - *Output:* Normalized 1D NumPy array (float32).
2. **Module 2: Acoustic Quality Gatekeeper** *(Planned)*
    - *Processing:* Calculates RMS energy and Signal-to-Noise Ratio (SNR) to classify incoming audio into "good", "degraded", or "insufficient". Short-circuits execution if audio is unreadable.
3. **Module 3: Demographic Inference Engine** *(Planned)*
    - *Processing:* Feeds the clean 16 kHz audio array into a pre-trained transformer model (e.g., audeering/wav2vec2-large-robust-24-ft-age-gender) running under isolated inference contexts.
4. **Module 4: Output Post-Processor & Serializer** *(Planned)*
    - *Processing:* Maps continuous model outputs into discrete categorical brackets (18-30, 31-45, 46-60, 60+), normalizes confidence scores, and formats timers.
5. **Module 5: API Layer & Telemetry** (app/main.py)
    - *Processing:* Exposes FastAPI endpoints (POST /analyze), validates payloads via Pydantic, and tracks execution metrics in milliseconds (processing_ms).

5. Implementation Progress & What We've Accomplished

- **Virtual Environment Setup:** Initialized an isolated Python virtual environment (venv) and managed dependencies (fastapi, uvicorn, librosa, soundfile, numpy, python-multipart).
- **Request Ingestion Layer Built:** Created the core app/main.py FastAPI server containing the POST /analyze endpoint.
- **In-Memory Stream Handling:** Successfully verified that incoming client files (such as audio/video note uploads via Swagger UI at http://localhost:8000/docs) are intercepted, logged, and securely loaded entirely into RAM as raw binary bytes without touching the local disk.